

frank_charlie_pset1

April 14, 2026

Charlie Frank

1 Homework 1

Power Systems Optimization

by Jesse D. Jenkins and Michael R. Davidson (last updated: September 14, 2022)

This Notebook will walk you through defining a simple transport flow model and then ask you to interact with the solutions and modify to model to add additional constraints...

1.1 Setting up the model

1.1.1 Load packages

```
[1]: using JuMP
      using HiGHS
      using DataFrames
      using CSV
```

1.1.2 Define sets

We will define two sets, both as arrays of strings

Production plants, P

```
[2]: P=["trenton", "newark"] # production plants
```

```
[2]: 2-element Vector{String}:
      "trenton"
      "newark"
```

Markets for products, M

```
[3]: M=["newyork", "princeton", "philadelphia"] # markets for products
```

```
[3]: 3-element Vector{String}:
      "newyork"
      "princeton"
      "philadelphia"
```

Note that sets can also be defined over intervals (as in `i=1:10`) or numerical vectors (as in `x=[2, 4, 5, 11]`)

1.1.3 Define parameters

We'll make use of the defined sets as indexes for our parameters...

Plant production capacities

```
[4]: plants = DataFrame(plant=P, capacity=[350,650])
```

```
[4]:
```

	plant	capacity
	String	Int64
1	trenton	350
2	newark	650

Demand for products

Stored in a `DataFrame`

```
[5]: markets = DataFrame(
      market=M,
      demand=[325, 300, 275]
    )
```

```
[5]:
```

	market	demand
	String	Int64
1	newyork	325
2	princeton	300
3	philadelphia	275

A few different ways to index into our DataFrames to access parameters (all of the below are equivalent)

```
[6]: plants[plants.plant=="newark",:capacity] # option 1
```

```
[6]: 1-element Vector{Int64}:
      650
```

```
[7]: plants[plants.plant=="newark",:].capacity # option 2
```

```
[7]: 1-element Vector{Int64}:
      650
```

```
[8]: plants.capacity[plants.plant=="newark"] # option 3
```

```
[8]: 1-element Vector{Int64}:
      650
```

```
[9]: plants[:, :capacity][plants.plant=="newark"] # option 4
```

```
[9]: 1-element Vector{Int64}:  
     650
```

Note that DataFrame indexing returns an Array by default, in this case, a 1-element Array of type Int64 (64-bit integer), as indicated by `Array{Int64,1}` above.

To access the single Int64 value, append `[1]` to any of the above to reference the first (and only) element in this array.

```
[10]: plants.capacity[plants.plant=="newark"][1]
```

```
[10]: 650
```

```
[11]: typeof(plants.capacity[plants.plant=="newark"][1])
```

```
[11]: Int64
```

```
[12]: typeof(plants.capacity[plants.plant=="newark"])
```

```
[12]: Vector{Int64} (alias for Array{Int64, 1})
```

Distance from plants to markets

Stored in a JuMP `DenseAxisArray` with data array and symbolic references across each of our sets (plants and markets), converted to Symbols for referencing

```
[13]: # two dimensional symbolic DenseAxisArray, with from/to distance pairs  
distances = JuMP.Containers.DenseAxisArray(  
    [2.5 0.5 1.5;  
     0.5 1.5 3.5],  
    Symbol.(P),  
    Symbol.(M),  
)
```

```
[13]: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:  
      Dimension 1, [:trenton, :newark]  
      Dimension 2, [:newyork, :princeton, :philadelphia]  
And data, a 2×3 Matrix{Float64}:  
 2.5  0.5  1.5  
 0.5  1.5  3.5
```

A couple example references to our `DenseAxisArray` to access parameters...

```
[14]: distances[:trenton, :newyork] #example of distance references
```

```
[14]: 2.5
```

```
[15]: distances[:newark, :newyork] #example of distance references
```

```
[15]: 0.5
```

```
[16]: distances[Symbol(P[2]),Symbol(M[1])] # another way to find distance from newark_
      ↪to trenton
```

```
[16]: 0.5
```

```
[17]: distances[Symbol("newark"), Symbol("newyork")] # and a third...
```

```
[17]: 0.5
```

Costs of transport

```
[18]: freight_cost = 90 # Cost of freight shipment per unit of distance
```

```
[18]: 90
```

1.1.4 Create model

(and specify the HiGHS solver)

```
[19]: transport = Model(HiGHS.Optimizer);
```

1.1.5 Define variables

Quantities of product to transport from plant $p \in P$ to market $m \in M$

```
[20]: @variable(transport, X[P,M] >= 0)
```

```
[20]: 2-dimensional DenseAxisArray{VariableRef,2,...} with index sets:
```

```
Dimension 1, ["trenton", "newark"]
```

```
Dimension 2, ["newyork", "princeton", "philadelphia"]
```

```
And data, a 2×3 Matrix{VariableRef}:
```

```
 X[trenton,newyork]  X[trenton,princeton]  X[trenton,philadelphia]
```

```
 X[newark,newyork]   X[newark,princeton]   X[newark,philadelphia]
```

Example reference to single quantity decision variable, the quantity shipped from Newark to Philadelphia:

```
[21]: X["newark","philadelphia"]
```

```
[21]:  $X_{newark,philadelphia}$ 
```

1.1.6 Define constraints

Supply capacity constraint

```
[22]: @constraint(transport, cSupply[p in P],
      sum(X[p,m] for m in M)
      <= plants.capacity[plants.plant.==p][1])
```

```
[22]: 1-dimensional DenseAxisArray{ConstraintRef{Model,
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
MathOptInterface.LessThan{Float64}}, ScalarShape},1,...} with index sets:
    Dimension 1, ["trenton", "newark"]
And data, a 2-element Vector{ConstraintRef{Model,
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
MathOptInterface.LessThan{Float64}}, ScalarShape}}:
    cSupply[trenton] : X[trenton,newyork] + X[trenton,princeton] +
X[trenton,philadelphia] <= 350
    cSupply[newark] : X[newark,newyork] + X[newark,princeton] +
X[newark,philadelphia] <= 650
```

Demand balance constraint

Ensure all demand is satisfied at each market

```
[23]: @constraint(transport, cDemand[m in M],
    sum(X[p,m] for p in P)
    >= markets.demand[markets.market.==m][1])
```

```
[23]: 1-dimensional DenseAxisArray{ConstraintRef{Model,
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
MathOptInterface.GreaterThan{Float64}}, ScalarShape},1,...} with index sets:
    Dimension 1, ["newyork", "princeton", "philadelphia"]
And data, a 3-element Vector{ConstraintRef{Model,
MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64},
MathOptInterface.GreaterThan{Float64}}, ScalarShape}}:
    cDemand[newyork] : X[trenton,newyork] + X[newark,newyork] >= 325
    cDemand[princeton] : X[trenton,princeton] + X[newark,princeton] >= 300
    cDemand[philadelphia] : X[trenton,philadelphia] + X[newark,philadelphia] >= 275
```

1.1.7 Define objective function

Minimize total cost of transport to satisfy all demand.

First we'll define an expression for total cost of shipments...

```
[24]: @expression(transport, eCost,
    sum(freight_cost*distances[Symbol(p),Symbol(m)]*X[p,m]
    for p in P, m in M)
    )
```

```
[24]: 225Xtrenton,newyork + 45Xtrenton,princeton + 135Xtrenton,philadelphia + 45Xnewark,newyork +
135Xnewark,princeton + 315Xnewark,philadelphia
```

Now we'll minimize this total cost

```
[25]: @objective(transport, Min, eCost)
```

```
[25]: 225Xtrenton,newyork + 45Xtrenton,princeton + 135Xtrenton,philadelphia + 45Xnewark,newyork +
135Xnewark,princeton + 315Xnewark,philadelphia
```

1.2 Interact with the model

(a) Now let's solve the model. In the blank cell below, enter the command for JuMP to solve a model and run the cell

```
[31]: optimize!(transport)
```

```
LP has 5 rows; 6 cols; 12 nonzeros
Coefficient ranges:
  Matrix  [1e+00, 1e+00]
  Cost    [4e+01, 3e+02]
  Bound   [0e+00, 0e+00]
  RHS     [3e+02, 6e+02]
Solving LP with useful basis so presolve not used
```

```
Model status      : Optimal
Objective value    : 8.5500000000e+04
P-D objective error : 0.0000000000e+00
HiGHS run time    : 0.00
```

(b) You've got a solution. Now query the objective function in the empty cell below and save it to a variable (name of your choice)

```
[32]: cost_total = objective_value(transport)
```

```
[32]: 85500.0
```

(c) Now query and save the optimal solution for X (the decisions about shipment quantities from plant to market) to an Array or DataFrame

```
[33]: X_optimal = [value(X[p,m]) for p in P, m in M]
```

```
[33]: 2×4 Matrix{Float64}:
 0.0  75.0  275.0  0.0
325.0 225.0   0.0  50.0
```

(d) Please interpret your results by writing an explanation in the markdown cell below.

Which facility or facilities supplies the most demand in New York? Does this result make sense? Why?

Which facility or facilities supplies the most demand in Philadelphia? Does this result make sense? Why?

Which facility or facilities supplies the demand in Princeton? Does this result make sense? Why?

For New York, all 325 units are shipped out of Newark, this makes sense since the cost to ship from there to New York is 45 per unit, while Trenton is significantly more expensive.(225 per unit) Simple economics show that Newark is the correct and optimal choice.

For Philadelphiia we see that all 275 units come from Trenton. This is also a simple economics problem since the prices from Trenton are 135 per unit while Newark is 315 per unit. This makes Trenton the sensible optimal solution.

Lastly, Princeton, while it could be cheaper to have all of Princetons demand met by the cheaper Trenton (45 per unit) facility, the constraints of this problem limit the results. Since Princetons most expensive option is Philadelphia's cheapest, we served all of Philly's demand with Trenton. Trenton has a capacity limit, meaning only part of Princetons demand can be met by its remaining capacity. The rest of Princetons demand must be met by Newark, which is 135 per unit, but within the 3 city optimization problem, this is the best solution for all 3 cities.

(e) A new market in New Brunswick appears, with a demand for 50 units. It is located 1.0 units away from both plants. Add this market to the model and solve again.

```
[34]: M = ["newyork", "princeton", "philadelphia", "newbrunswick"]
markets = DataFrame(
    market = M,
    demand = [325, 300, 275, 50]
)
distances = JuMP.Containers.DenseAxisArray(
    [2.5 0.5 1.5 1.0;
     0.5 1.5 3.5 1.0],
    Symbol.(P),
    Symbol.(M),
)
transport2 = Model(HiGHS.Optimizer)
@variable(transport2, X[P, M] >= 0)
@constraint(transport2, cSupply[p in P],
    sum(X[p,m] for m in M) <= plants.capacity[plants.plant .== p][1]
)
@constraint(transport2, cDemand[m in M],
    sum(X[p,m] for p in P) >= markets.demand[markets.market .== m][1]
)
@expression(transport2, eCost,
    sum(freight_cost * distances[Symbol(p), Symbol(m)] * X[p,m]
        for p in P, m in M)
)
@objective(transport2, Min, eCost)
optimize!(transport2)
```

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 6 rows; 8 cols; 16 nonzeros

Coefficient ranges:

Matrix	[1e+00, 1e+00]
Cost	[4e+01, 3e+02]
Bound	[0e+00, 0e+00]
RHS	[5e+01, 6e+02]

Presolving model

6 rows, 8 cols, 16 nonzeros 0s

6 rows, 8 cols, 16 nonzeros 0s

Presolve reductions: rows 6(-0); columns 8(-0); nonzeros 16(-0) - Not reduced

```

Problem not reduced by presolve: solving the LP
Using dual simplex solver
  Iteration      Objective      Infeasibilities num(sum)
        0      0.0000000000e+00 Pr: 4(950) 0.0s
        5      9.0000000000e+04 Pr: 0(0) 0.0s

```

```

Model status      : Optimal
Simplex iterations: 5
Objective value    : 9.0000000000e+04
P-D objective error: 0.0000000000e+00
HiGHS run time    : 0.00

```

(f) What is new optimal solution?

```

[35]: cost_total_brunswick = objective_value(transport2)
      println(cost_total_brunswick)
      X_optimal_brunswick = [value(X[p,m]) for p in P, m in M]

```

90000.0

```

[35]: 2×4 Matrix{Float64}:
      0.0  75.0  275.0   0.0
     325.0 225.0   0.0  50.0

```

(g) Interpret this result in the markdown cell below. Which facility or facilities supplies the demand in New Brunswick? Does this result make sense? Why?

New Brunswicks new demand is met by Newark exclusively. This makes sense since the Trenton and Newark facilities are equidistant from New Brunswick, thus making neither facility a cheaper or better option. Since Newark still has remaining shipping capacity in the original optimization problem, we can use its facility to serve New Brunswicks demand without modifying the other optimal values. Since the price for serving New Brunswicks demand is essentially fixed, just working it into keeping the original optimization with the 3 cities as cheap as possible makes the most sense, and that is what the results show.